

Implementation Screen snapshot

Min cost, min time implementation based on your requirements

1, Storage

Amazon S3

>

Buckets

>

hc-pipeline-demo

hc-pipeline-demo

Info

Objects

Metadata

Properties

Permissions

Objects (5)

Objects are the fundamental entities stored in Amazon S3. You can use [A](#)

Find objects by prefix

☐	Name	Type
☐	 athena-results/	Folder
☐	 bronze/	Folder
☐	 gold/	Folder
☐	 raw/	Folder
☐	 silver/	Folder

Amazon S3

>

Buckets

>

hc-pipeline-demo

>

raw/

raw/

Objects

Properties

Objects (1)

Copy S3 URI

Copy URL

Download

Open

Delete

Action

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant

Find objects by prefix

☐	Name	Type	Last modified	Size
☐	 DETask.xlsx	xlsx	May 1, 2026, 20:14:53 (UTC-05:00)	664.4 KB

bronze/

Objects Properties

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant

☐	Find objects by prefix				
☐	Name	Type	Last modified	Size	
☐	 claims_bronze_20260502_031803.parquet	parquet	May 1, 2026, 22:18:04 (UTC-05:00)	172.8 KB	

silver/

Objects Properties

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant

☐	Find objects by prefix				
☐	Name	Type	Last modified	Size	
☐	 claims_silver_20260502_172631.parquet	parquet	May 2, 2026, 12:26:33 (UTC-05:00)	214.6 KB	

gold/

Objects Properties

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#)

☐	Find objects by prefix				
☐	Name	Type			
☐	 iceberg/	Folder			
☐	 staging/	Folder			

staging/

Objects

Properties

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant

Find objects by prefix

☐	Name	Type	Last modified	Size
☐	data_20260502_181431.parquet	parquet	May 2, 2026, 13:14:34 (UTC-05:00)	214.6 KB

iceberg/

Objects

Properties

Objects (3)

Objects are the fundamental entities stored in Amazon S3. You can use [Ama](#)

Find objects by prefix

☐	Name	Type
☐	dim_procedure/	Folder
☐	dim_provider/	Folder
☐	fact_claims/	Folder

fact_claims/

Objects

Properties

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3](#)

Find objects by prefix

☐	Name	Type
☐	 data/	Folder
☐	 metadata/	Folder

metadata/

Objects

Properties

Objects (4)

Copy S3 URIDownloadOpen iDeleteActions

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant t

Find objects by prefix

☐	Name	Type	Last modified	Size
☐	 00000-722c95be-699a-4ffd-9896-1f6cb0d525e9.metadata.json	json	May 2, 2026, 13:14:39 (UTC-05:00)	2.5 KB
☐	 00001-3cd8934d-c28c-4ec9-8ed7-3a817c64738b.metadata.json	json	May 2, 2026, 13:14:48 (UTC-05:00)	3.5 KB
☐	 b33b6415-b31e-4426-93ac-509429606df9-m0.avro	avro	May 2, 2026, 13:14:47 (UTC-05:00)	9.7 KB
☐	 snap-2662624071159128714-1-b33b6415-b31e-4426-93ac-509429606df9.avro	avro	May 2, 2026, 13:14:47 (UTC-05:00)	4.2 KB

data/

Objects Properties

Objects (12)



Copy

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permission.

Find objects by prefix

☐	Name	Type
☐	_C5kQ/	Folder
☐	0BouwW/	Folder
☐	73hoUQ/	Folder
☐	9C_YNw/	Folder
☐	d7Sf5A/	Folder
☐	jy2FrA/	Folder
☐	nk0eww/	Folder
☐	O5qmhA/	Folder
☐	RVXs6g/	Folder
☐	t8NiKw/	Folder
☐	wUnUJw/	Folder
☐	YLVwFg/	Folder

month=10/

Objects Properties

Objects (1)



Copy S3 URI



Copy URL



Download



Open



Delete



Action

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permission.

Find objects by prefix

☐	Name	Type	Last modified	Size
☐	20260502_181444_00088_mtv83-74f34718-f87c-478f-af2b-d26e480bb356.parquet	parquet	May 2, 2026, 13:14:47 (UTC-05:00)	11.9 KB

2, Lambda computing

≡ [Lambda](#) > Functions

Functions (3) Last fetched 5/2/2026, 4:11:44 PM [Actions](#) [Cr](#)

Q Search by attributes or search by keyword

☐	Function name	Description	Package type	Runtime	Type	Last modified
☐	gold-transformation	-	Zip	Python 3.12	Standard	3 hours ago
☐	bronze-ingestion	-	Zip	Python 3.12	Standard	19 hours ago
☐	silver-clean	-	Zip	Python 3.12	Standard	4 hours ago

Use built-in layer

Layers (1) [Info](#) Last fetched 5/2/2026, 4:12:16 PM [1](#)

Q Search by attributes or search by keyword

Merge order	Name	Layer version	Compatible runtimes	Compatible architectures	Version ARN
1	AWSSDKPandas-Python312	24	python3.12	x86_64	arn:aws:lambda:us-east-1:336392948345:layer:AWSSDKPandas-Python312:24

Use different role, gold-lambda use gold-lambda-role; brone and silver use same role called “[bronze-ingestion-role-b82ljmie](#)”

≡ [Lambda](#) > [Functions](#) > gold-transformation

[Code](#) [Test](#) [Monitor](#) [Configuration](#) [Aliases](#) [Versions](#)

Configuration

- General configuration
- Triggers
- Permissions**
- Destinations
- Function URL
- Environment variables
- Tags
- VPC
- RDS databases
- Monitoring and operations tools
- Concurrency and recursion detection
- Asynchronous invocation

Execution role [View role document](#)

Role name
[gold-lambda-role](#)

Resource summary
To view the resources and actions that your function has permission to access, choose a service.

[Amazon Athena](#)
4 actions 1 resource

By action **By resource**

Resource	Actions
All resources	Allow: athena:StartQueryExecution Allow: athena:GetQueryExecution Allow: athena:GetQueryResults Allow: athena:StopQueryExecution

Lambda obtained this information from the following policy statements:

- Managed policy gold-lambda-policy, statement 2

3, IAM

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- Roles
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings

Roles (5) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assigned to users, groups, or other roles.

Search

☐	Role name	Trusted entities	Last activity
☐	AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Service-Linked Role)	6 hours ago
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Linked Role)	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linked Role)	-
☐	bronze-ingestion-role-b82ljmie	AWS Service: lambda	6 hours ago
☐	gold-lambda-role	AWS Service: lambda	6 hours ago

4, Glue Data Catalog,

AWS Glue > **Databases** > healthcare_gold

healthcare_gold

Last updated (UTC) May 2, 2026 at 21:17:36

Edit Delete

Database properties

Name	Description	Location	Created on (UTC)
healthcare_gold	Healthcare claims Gold layer - Iceberg tables	-	May 2, 2026 at 14:34:11

Tables (4)

View and manage all available tables.

Last updated (UTC) May 2, 2026 at 18:17:05

Delete Add tables using crawler Add table

Filter tables

Name	Database	Location	Classification	Deprecated	View data	Data quality	Column statistics
dim_procedure	healthcare_gold	s3://hc-pipeline-demo/gold/iceberg/dim_procedure	-	-	Table data	View data quality	View statistics
dim_provider	healthcare_gold	s3://hc-pipeline-demo/gold/iceberg/dim_provider	-	-	Table data	View data quality	View statistics
fact_claims	healthcare_gold	s3://hc-pipeline-demo/gold/iceberg/fact_claims	-	-	Table data	View data quality	View statistics
staging_claims	healthcare_gold	s3://hc-pipeline-demo/gold/staging	-	-	Table data	View data quality	View statistics

AWS Glue > **Databases**

Databases (4)

Last updated (UTC) May 3, 2026 at 02:05:44

Edit Delete Add database

A database is a set of associated table definitions, organized into a logical group.

Choose catalog

899957567554

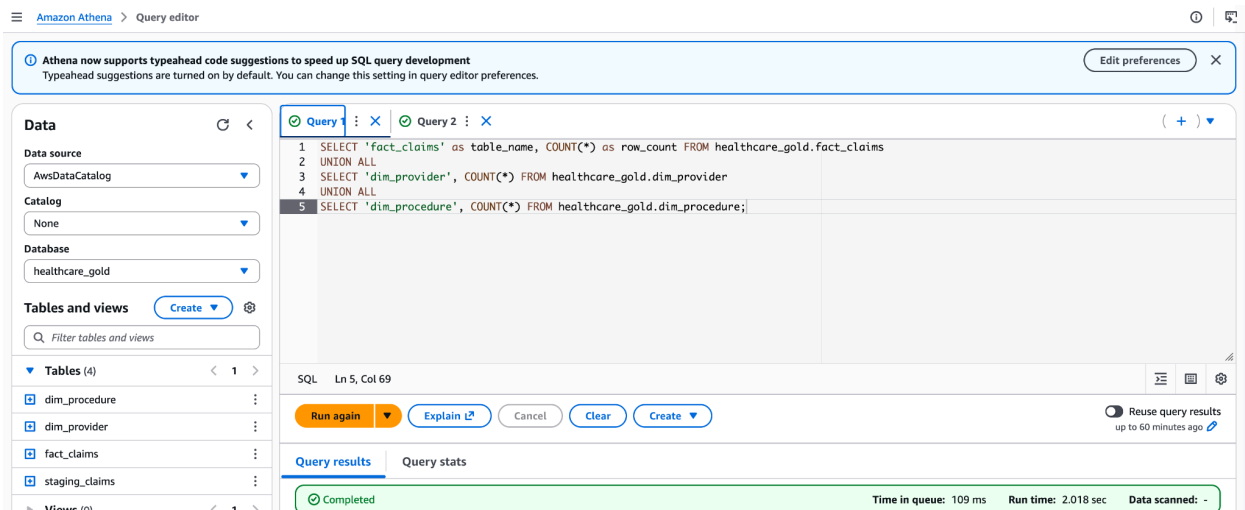
Filter databases

☐	Name	Description	Location URI	Source catalog	Created on (UTC)
☐	default	-	-	899957567554	May 2, 2026 at 14:51:28
☐	healthcare_gold	Healthcare claims Gold layer - Iceberg tables	-	899957567554	May 2, 2026 at 14:34:11
☐	healthcare_silver	-	-	899957567554	May 2, 2026 at 21:57:07
☐	sagemaker_sample_db	-	-	899957567554	May 2, 2026 at 22:26:20

AWS Glue supports multiple table optimization options to enhance the management and performance of Apache Iceberg tables used by the AWS analytical engines and ETL jobs. These optimizers provide efficient storage utilization, improved query performance, and effective data management; Catalog-level optimization configuration is available through the Lake Formation console and using the AWS Glue `UpdateCatalog` API operation. You can enable or disable compaction, snapshot retention, and orphan file

deletion optimizers for individual Iceberg tables in the Data Catalog using the AWS Glue console, AWS CLI, or AWS Glue API operations.

5, Athena –semantic layer for tableau/QuickSight/PowerBI or other BI tools



6, Views for a Complete Semantic Layer

```
-- vw_monthly_costs.sql --- Monthly charge summary by in-network status
```

```
CREATE OR REPLACE VIEW healthcare_gold.vw_monthly_costs AS
SELECT
  DATE_TRUNC('month', f.received_date) AS month,
  f.in_network,
  p.state,
  COUNT(*) AS claim_count,
  SUM(f.charge_amt) AS total_charged,
  SUM(f.allowed_amt) AS total_allowed,
  SUM(f.discount_amt) AS total_discount,
  ROUND(AVG(f.discount_amt / NULLIF(f.charge_amt,0)) * 100, 2) AS
avg_discount_pct
FROM healthcare_gold.fact_claims f
JOIN healthcare_gold.dim_provider p USING (provider_npi)
GROUP BY 1, 2, 3;
```

```
-- vw_provider_kpis.sql --- Provider performance metrics
```



```

CREATE OR REPLACE VIEW healthcare_gold.vw_provider_kpis AS
SELECT
    p.provider_name,
    p.city, p.state,
    COUNT(*) AS total_claims,
    SUM(f.charge_amt) AS total_charged,
    ROUND(AVG(f.charge_amt), 2) AS avg_charge,
    SUM(CASE WHEN f.in_network THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS
in_network_pct
FROM healthcare_gold.fact_claims f
JOIN healthcare_gold.dim_provider p USING (provider_npi)
GROUP BY 1, 2, 3;

```

```

-- Tracks the volume and value of COVID-19 related procedures (U0003, U0004,
U0005, 87635, 87426)
CREATE OR REPLACE VIEW healthcare_gold.vw_covid19_tests AS
SELECT
    DATE_TRUNC('month', f.received_date) AS month,
    f.claim_code,
    dp.procedure_desc,
    COUNT(*) AS test_count,
    SUM(f.charge_amt) AS total_charge,
    SUM(f.allowed_amt) AS total_allowed,
    ROUND(AVG(f.discount_amt), 2) AS avg_discount_per_test
FROM healthcare_gold.fact_claims f
JOIN healthcare_gold.dim_procedure dp ON f.claim_code = dp.claim_code
WHERE f.claim_code IN ('U0003', 'U0004', 'U0005', '87635', '87426', 'G2023', 'G2024')
GROUP BY 1, 2, 3
ORDER BY month DESC, test_count DESC;

```

```

-- Identifies which labs performed most COVID tests and their network status
CREATE OR REPLACE VIEW healthcare_gold.vw_covid19_providers AS
SELECT
    p.provider_name,
    p.state,
    f.in_network,
    f.claim_code,
    COUNT(*) AS tests_performed,
    SUM(f.charge_amt) AS total_charged

```

```

FROM healthcare_gold.fact_claims f
JOIN healthcare_gold.dim_provider p ON f.provider_npi = p.provider_npi
WHERE f.claim_code LIKE 'U%' OR f.claim_code LIKE '87%' OR f.claim_code =
'G2023'
GROUP BY 1, 2, 3, 4
ORDER BY tests_performed DESC;

```

-- Compares costs and discounts between in-network and out-of-network claims

```

CREATE OR REPLACE VIEW healthcare_gold.vw_network_impact AS

```

```

SELECT

```

```

    f.in_network,
```

```
    COUNT(*) AS total_claims,
```

```
    SUM(f.charge_amt) AS total_charge,
```

```
    SUM(f.allowed_amt) AS total_allowed,
```

```
    SUM(f.discount_amt) AS total_discount,
```

```
    ROUND(AVG(f.discount_amt / NULLIF(f.charge_amt,0)) * 100, 2) AS
```

```
avg_discount_rate
```

```

FROM healthcare_gold.fact_claims f

```

```

GROUP BY 1;

```

-- Identifies claims where the discount was unusually high (e.g., >50% of charge)

```

CREATE OR REPLACE VIEW healthcare_gold.vw_high_discount_alerts AS

```

```

SELECT

```

```
    f.claim_item_id,
```

```
    dp.procedure_desc,
```

```
    f.charge_amt,
```

```
    f.allowed_amt,
```

```
    f.discount_amt,
```

```
    ROUND(f.discount_amt / f.charge_amt * 100, 2) AS discount_pct,
```

```
    f.received_date,
```

```
    p.provider_name
```

```

FROM healthcare_gold.fact_claims f

```

```

JOIN healthcare_gold.dim_provider p ON f.provider_npi = p.provider_npi

```

```

JOIN healthcare_gold.dim_procedure dp ON f.claim_code = dp.claim_code

```

```

WHERE f.charge_amt > 0

```

```
    AND (f.discount_amt / f.charge_amt) > 0.5 -- Discounts greater than 50%
```

```

ORDER BY discount_pct DESC;

```

-- High-level financial dashboard for executives

-- vw_financial_summary.sql --- High-level financial dashboard for executives

```

CREATE OR REPLACE VIEW healthcare_gold.vw_financial_summary AS
SELECT
    DATE_TRUNC('month', received_date) AS month,
    COUNT(DISTINCT claimant_id) AS unique_patients,
    SUM(units) AS total_service_units,
    SUM(charge_amt) AS gross_revenue,
    SUM(allowed_amt) AS net_revenue,
    SUM(charge_amt - allowed_amt) AS total_contractual_adjustments,
    ROUND(SUM(charge_amt - allowed_amt) / NULLIF(SUM(charge_amt), 0) * 100, 2)
AS discount_rate_pct
FROM healthcare_gold.fact_claims
GROUP BY 1
ORDER BY 1 DESC;

```

```

-- Ranks CPT/HCPCS codes by volume and revenue
CREATE OR REPLACE VIEW healthcare_gold.vw_top_procedures AS
SELECT
    dp.claim_code,
    dp.procedure_desc,
    COUNT(*) AS times_billed,
    SUM(f.units) AS total_units,
    SUM(f.charge_amt) AS total_charges,
    RANK() OVER (ORDER BY COUNT(*) DESC) AS rank_by_volume
FROM healthcare_gold.fact_claims f
JOIN healthcare_gold.dim_procedure dp ON f.claim_code = dp.claim_code
GROUP BY 1, 2
ORDER BY times_billed DESC;

```

```

-- Since DIAG_CODE_2-5 were unpivoted to a bridge table (conceptually),
-- this view simulates joining primary diagnosis to facts.
-- Note: You would need a dim_diagnosis table for this.
-- Alternative: Query Silver layer directly for diagnosis analysis
-- This works because Silver retains all DIAG_CODE columns
CREATE OR REPLACE VIEW healthcare_gold.vw_diagnosis_silver AS
SELECT
    CAST(claim_item_id AS BIGINT) AS claim_item_id,
    CAST(claimant_id AS BIGINT) AS claimant_id,
    claim_code,
    provider_npi,

```

```

diag_code_1 AS primary_diagnosis,
diag_code_2 AS secondary_diagnosis_1,
diag_code_3 AS secondary_diagnosis_2,
diag_code_4 AS secondary_diagnosis_3,
diag_code_5 AS secondary_diagnosis_4,
charge_amt,
allowed_amt,
received_date
FROM healthcare_silver.claims_silver;

```

*****now we use silver for above views, run this first then run above view

```

CREATE EXTERNAL TABLE IF NOT EXISTS healthcare_silver.claims_silver (
  claimant_id STRING,
  claim_item_id STRING,
  type STRING,
  received_date STRING,
  charge_amt STRING,
  allowed_amt STRING,
  diag_code_1 STRING,
  diag_code_2 STRING,
  diag_code_3 STRING,
  diag_code_4 STRING,
  diag_code_5 STRING,
  claim_code STRING,
  proc_desc STRING,
  claim_code_modifier STRING,
  claim_code_modifier_2 STRING,
  units STRING,
  oi_in_network STRING,
  service_provider STRING,
  service_address_3 STRING,
  service_address_2 STRING,
  provider_npi STRING,
  city STRING,
  state STRING,
  is_valid BOOLEAN
)

```

```
ROW FORMAT SERDE
'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe'
LOCATION 's3://hc-pipeline-demo/silver/';
```

```
-- Tracks patient activity without exposing PII (CLAIMANT_ID masked for analysts)
CREATE OR REPLACE VIEW healthcare_gold.vw_patient_activity AS
SELECT
  MD5(TO_UTF8(CAST(f.claimant_id AS VARCHAR))) AS hashed_patient_id,
  COUNT(f.claim_item_id) AS total_claims,
  SUM(f.charge_amt) AS total_patient_charges,
  SUM(f.allowed_amt) AS total_patient_allowed,
  DATE_TRUNC('month', MIN(f.received_date)) AS first_service_date,
  DATE_TRUNC('month', MAX(f.received_date)) AS last_service_date
FROM healthcare_gold.fact_claims f
GROUP BY MD5(TO_UTF8(CAST(f.claimant_id AS VARCHAR)))
HAVING COUNT(*) > 1
ORDER BY total_patient_charges DESC;
```

```
-- Compares average costs by state
CREATE OR REPLACE VIEW healthcare_gold.vw_geo_cost_variation AS
SELECT
  p.state,
  COUNT(*) AS claim_count,
  ROUND(AVG(f.charge_amt), 2) AS avg_charge,
  ROUND(AVG(f.allowed_amt), 2) AS avg_allowed,
  ROUND(AVG(f.discount_amt), 2) AS avg_discount
FROM healthcare_gold.fact_claims f
JOIN healthcare_gold.dim_provider p ON f.provider_npi = p.provider_npi
GROUP BY p.state
HAVING COUNT(*) > 10
ORDER BY avg_charge DESC;
```

```
-- Classifies services as 'COVID', 'Mental Health', 'Therapy', 'Surgery', 'Lab'
CREATE OR REPLACE VIEW healthcare_gold.vw_service_type_breakdown AS
```

```

SELECT
CASE
    WHEN claim_code IN ('U0003', 'U0004', 'U0005', '87635', '87426', 'G2023', 'G2024')
THEN 'COVID-19 Testing'
    WHEN claim_code BETWEEN '90832' AND '90899' THEN 'Psychiatry/Mental Health'
    WHEN claim_code BETWEEN '97110' AND '97599' THEN 'Therapy/Rehab'
    WHEN claim_code BETWEEN '10000' AND '69999' THEN 'Surgery/Procedure'
    ELSE 'Lab/Other'
END AS service_category,
DATE_TRUNC('month', received_date) AS month,
COUNT(*) AS volume,
SUM(charge_amt) AS revenue
FROM healthcare_gold.fact_claims
GROUP BY 1, 2
ORDER BY month, volume DESC;
-- Analyzes how many line items are billed per patient per day (helps detect billing
patterns)
CREATE OR REPLACE VIEW healthcare_gold.vw_claim_density AS
SELECT
    claimant_id,
    received_date,
    COUNT(*) AS line_items_per_day,
    SUM(charge_amt) AS daily_charge
FROM healthcare_gold.fact_claims
GROUP BY 1, 2
HAVING COUNT(*) > 5 -- Days with unusually high billing
ORDER BY line_items_per_day DESC;

-- Flags claims where the payer allowed $0 (denials or non-covered services)
CREATE OR REPLACE VIEW healthcare_gold.vw_zero_allowed_audit AS
SELECT
    f.claim_item_id,
    dp.procedure_desc,
    f.charge_amt,
    f.allowed_amt,
    f.received_date,
    p.provider_name,
    f.in_network
FROM healthcare_gold.fact_claims f
JOIN healthcare_gold.dim_provider p ON f.provider_npi = p.provider_npi

```

```
JOIN healthcare_gold.dim_procedure dp ON f.claim_code = dp.claim_code
WHERE f.allowed_amt = 0 AND f.charge_amt > 0
ORDER BY f.charge_amt DESC;
```

```
-- Provides a BI-accessible view of potential duplicates identified earlier
CREATE OR REPLACE VIEW healthcare_gold.vw_duplicate_suspicious AS
SELECT
  claimant_id,
  received_date,
  claim_code,
  charge_amt,
  COUNT(*) AS duplicate_count
FROM healthcare_gold.fact_claims
GROUP BY 1, 2, 3, 4
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC, charge_amt DESC;
```

```
-- Cumulative fiscal year summary
CREATE OR REPLACE VIEW healthcare_gold.vw_ytd_summary AS
SELECT
  claimant_id,
  EXTRACT(YEAR FROM received_date) AS year,
  SUM(charge_amt) AS ytd_charges,
  SUM(allowed_amt) AS ytd_allowed
FROM healthcare_gold.fact_claims
WHERE EXTRACT(MONTH FROM received_date) <= 6 -- Example: Q2 reporting
GROUP BY 1, 2;
```

The solution provides a foundation for the semantic layer. The 16 views above transform the Gold layer from a simple data model into a powerful, business-ready semantic layer that answers specific questions about COVID costs, provider performance, network leakage, fraud detection (high discounts), and clinical service lines.

ALL SQL were tested and run successfully

Tables and views

Create ▼



🔍 *Filter tables and views*

▼ Tables (4) < 1 >

 dim_procedure	⋮
 dim_provider	⋮
 fact_claims	⋮
 staging_claims	⋮

▼ Views (16) < 1 >

 vw_claim_density	⋮
 vw_covid19_providers	⋮
 vw_covid19_tests	⋮
 vw_diagnosis_silver	⋮
 vw_duplicate_suspicious	⋮
 vw_financial_summary	⋮
 vw_geo_cost_variation	⋮
 vw_high_discount_alerts	⋮
 vw_monthly_costs	⋮
 vw_network_impact	⋮

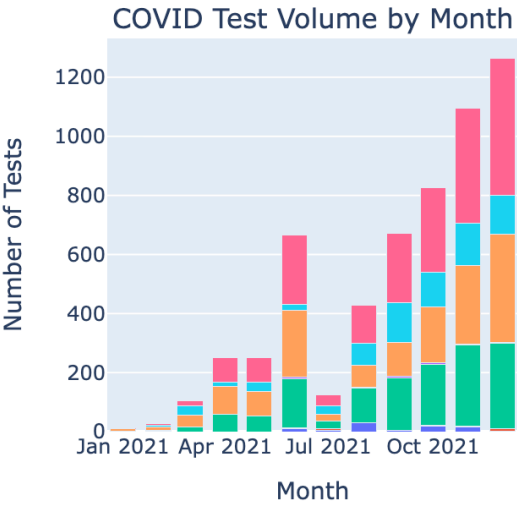
Using those views and create some Vizs as below, created by Sagemake notebook,



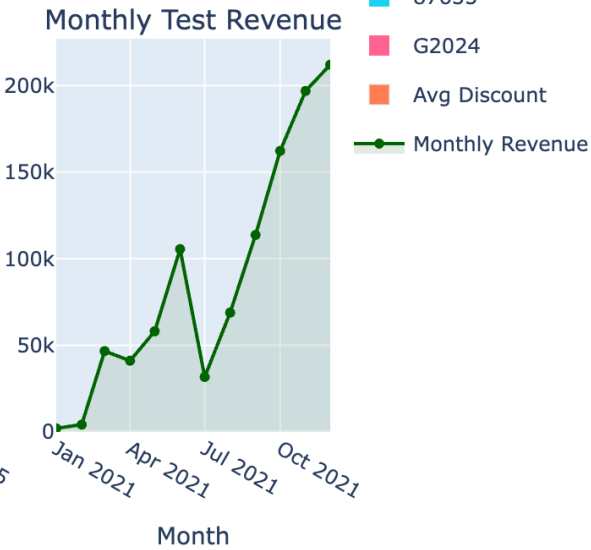
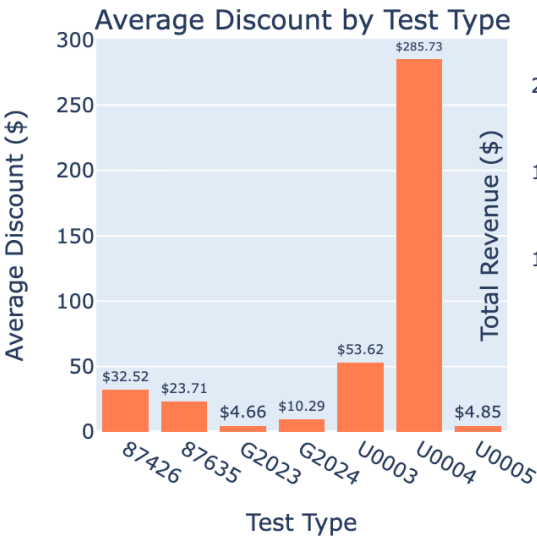
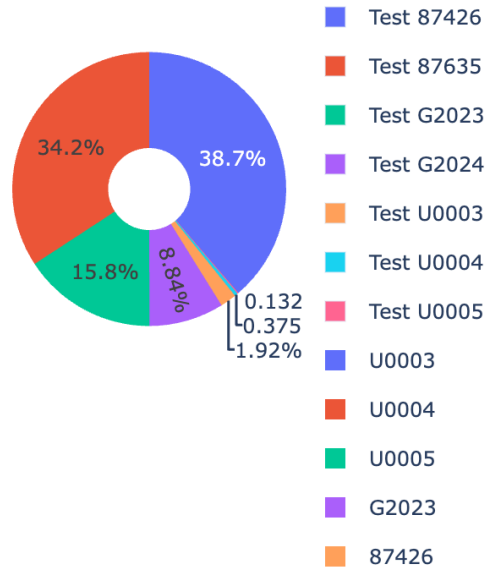
Healthcare Financial Dashboard



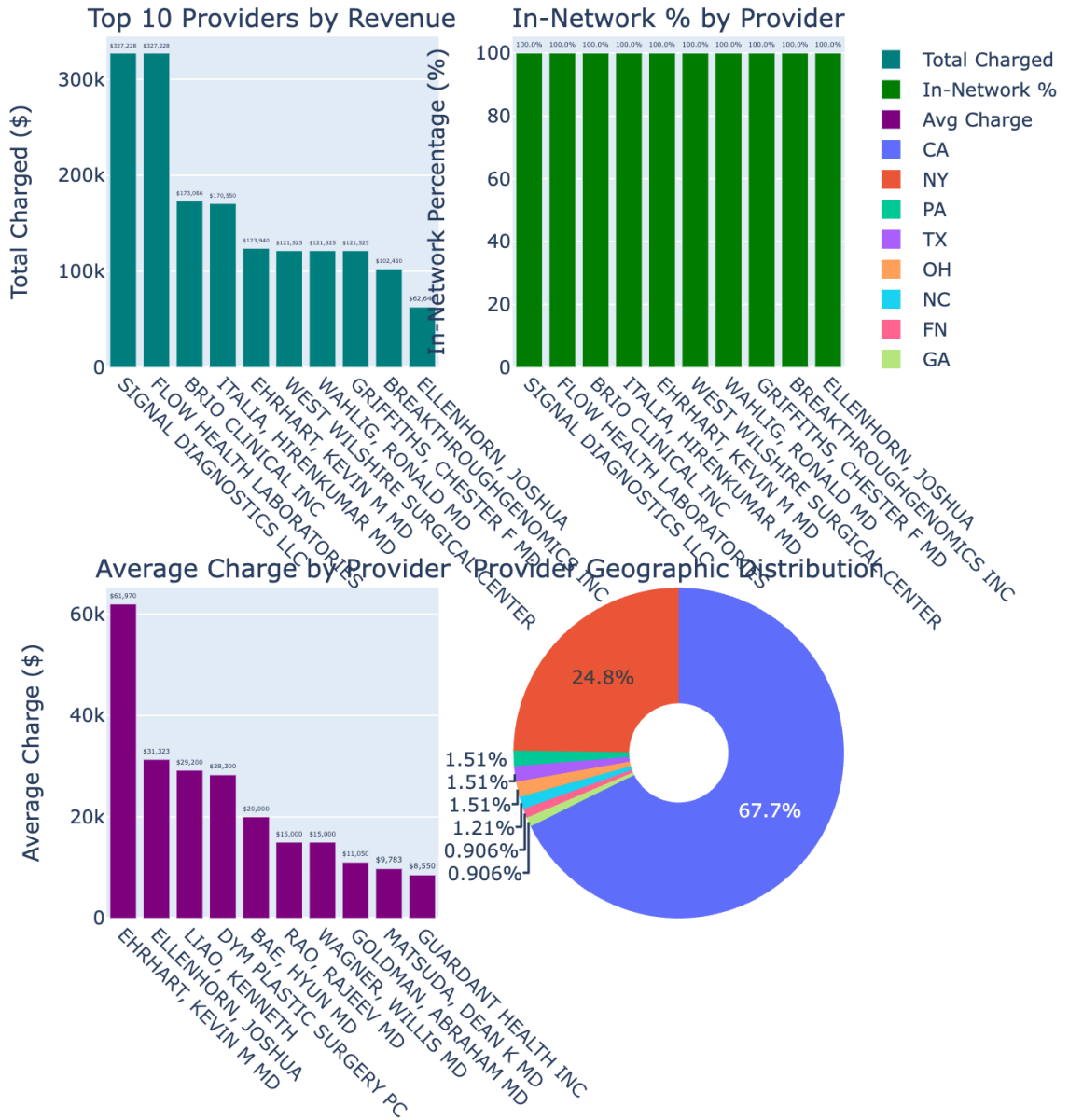
COVID-19 Testing Analytics



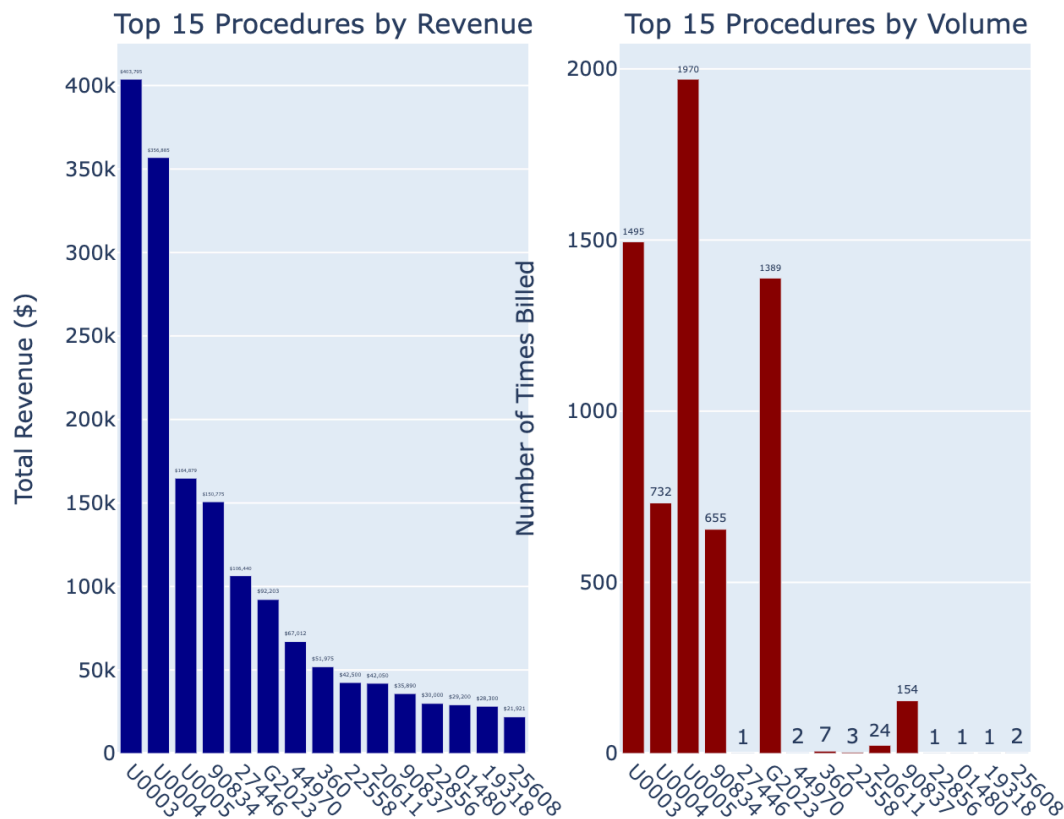
Revenue by Test Type



Provider Performance Analytics



Procedure Analysis



Procedure Analysis Summary

Total unique procedures: 431
Total revenue across all procedures: \$2,565,170.32
Total claims across all procedures: 8,381

🏆 Top 5 Procedures by Revenue:

U0003:
Revenue: \$403,795.46
Claims: 1,495
Units: 1,495

U0004:

Revenue: \$356,885.25

Claims: 732

Units: 732

U0005:

Revenue: \$164,879.25

Claims: 1,970

Units: 1,970

90834:

Revenue: \$150,775.00

Claims: 655

Units: 665

27446:

Revenue: \$106,440.00

Claims: 1

Units: 1

💰 Highest Revenue per Claim Procedures:

27446: \$106,440.00 per claim

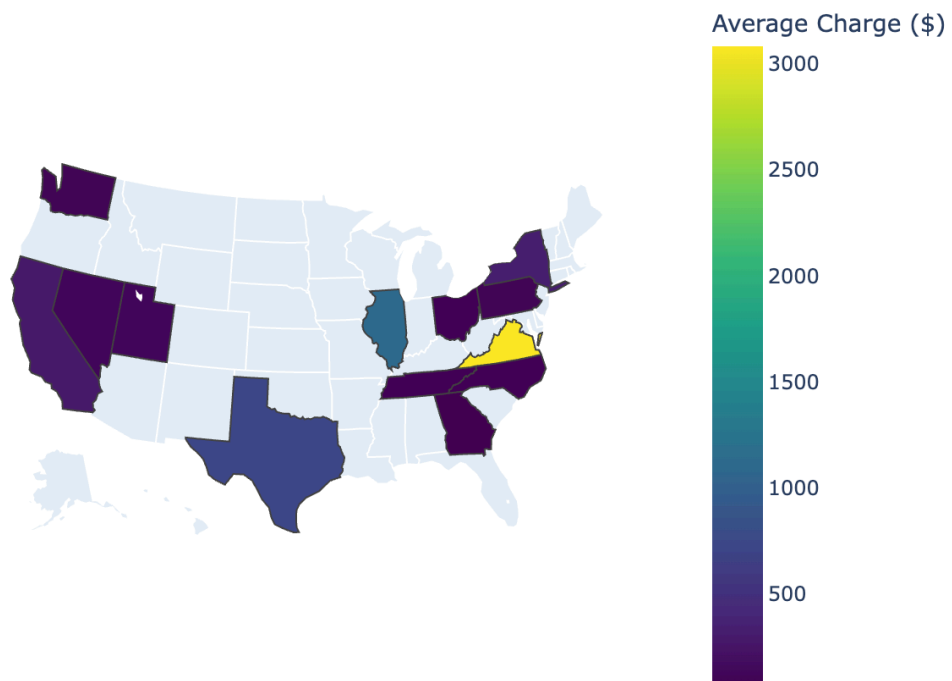
44970: \$33,505.90 per claim

22856: \$30,000.00 per claim

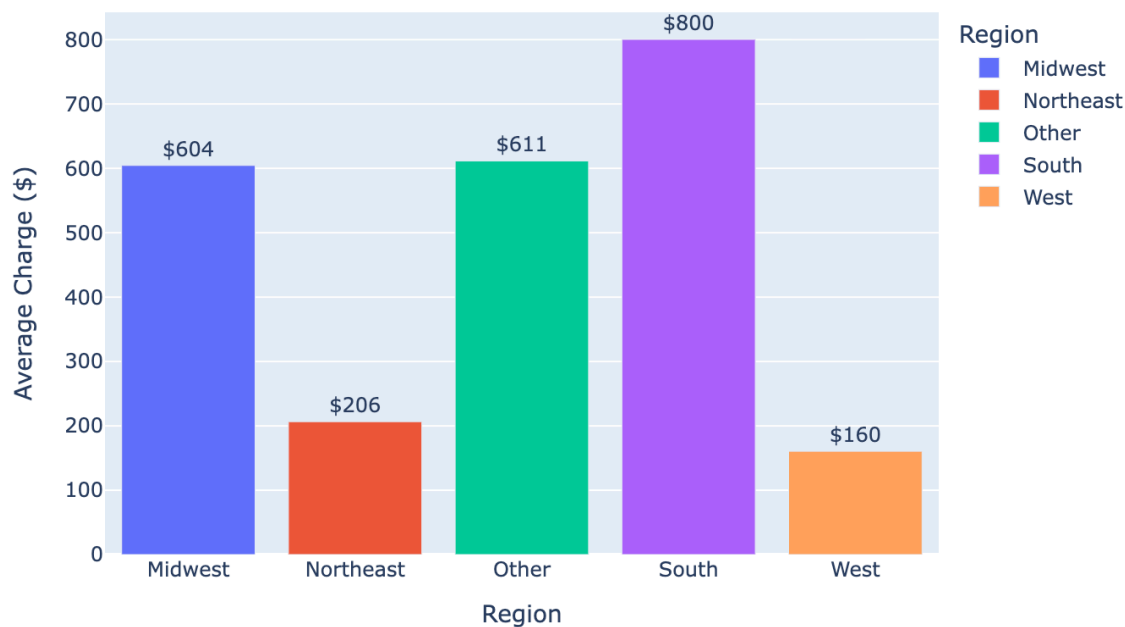
01480: \$29,200.00 per claim

19318: \$28,300.00 per claim

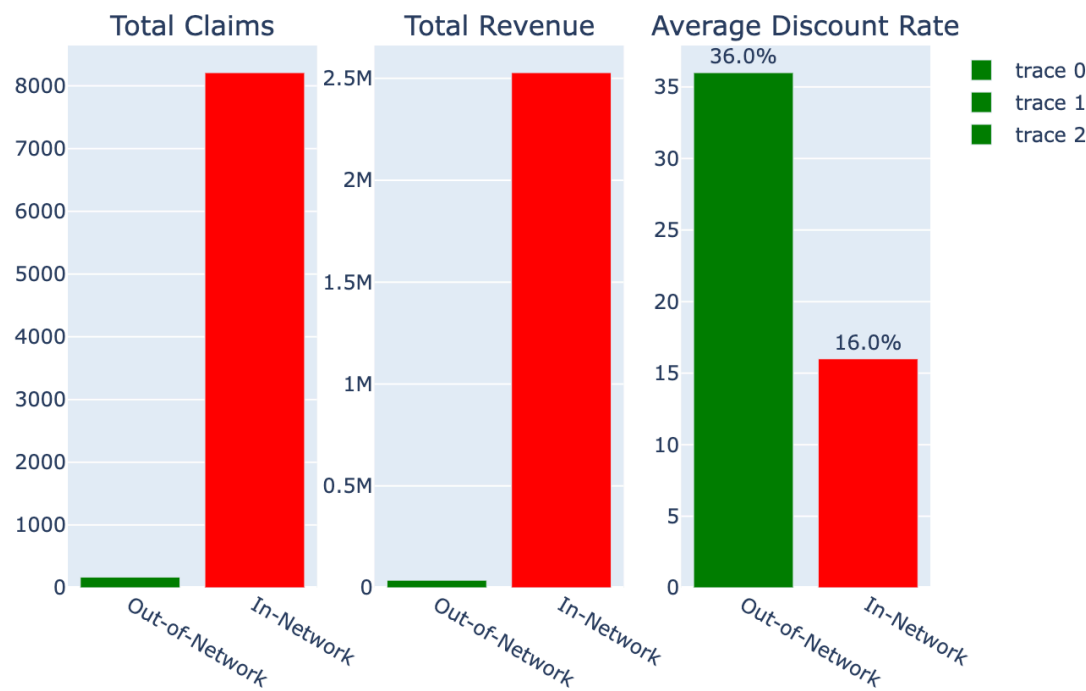
Average Charge by State



Average Charge by Region

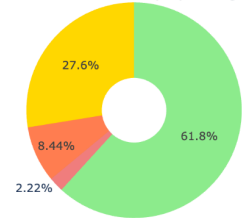


In-Network vs Out-of-Network Analysis

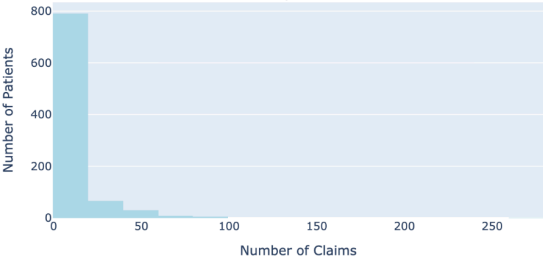


Patient Activity & Segmentation Analysis

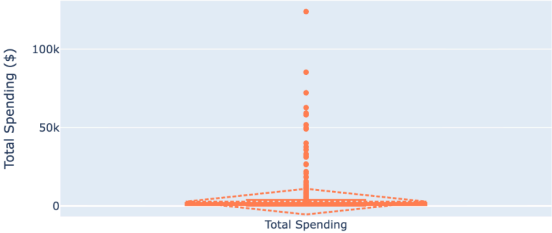
Patient Distribution by Spending Tier



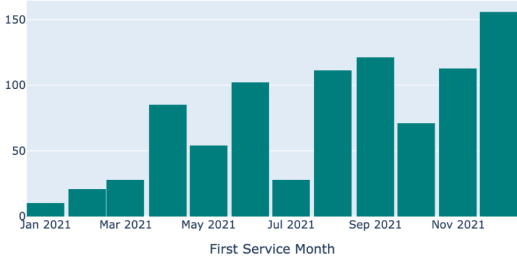
Claims per Patient



Total Patient Spending Distribution



First Service Month Distribution





Healthcare Analytics Executive Report

Generated on: 2026-05-03 01:07:17



Key Performance Indicators

\$2,565,170

Total Gross Revenue

\$1,629,859

Total Net Revenue

1,706

Unique Patients (Multi-Visit)

5,722

COVID-19 Tests Performed

\$1,043,112

COVID-19 Test Revenue

98.0%

In-Network Claims



COVID-19 Testing Summary

claim_code	test_count	total_charge	avg_discount_per_test
87426	86	20053.09	32.52
87635	23	3915.00	23.71
G2023	1389	92203.42	4.66
G2024	27	1381.00	10.29
U0003	1495	403795.46	53.62
U0004	732	356885.25	285.73
U0005	1970	164879.25	4.86

🏆 Top 10 Providers by Revenue

provider_name	total_charged	total_claims	in_network_pct
SIGNAL DIAGNOSTICS LLC	327228.00	2117	100.0
FLOW HEALTH LABORATORIES	327228.00	2117	100.0
BRIO CLINICAL INC	173066.25	174	100.0
ITALIA, HIRENKUMAR MD	170550.00	895	100.0
EHRHART, KEVIN M MD	123940.00	2	100.0
WEST WILSHIRE SURGICAL CENTER	121525.00	18	100.0
WAHLIG, RONALD MD	121525.00	18	100.0
GRIFFITHS, CHESTER F MD	121525.00	18	100.0
BREAKTHROUGHGENOMICS INC	102450.00	538	100.0
ELLENHORN, JOSHUA	62645.80	2	100.0

⚠️ High Discount Alerts (>50%)

procedure_desc	charge_amt	allowed_amt	discount_pct	provider_name
<NA>	45.00	0.0	100.0%	BROFFMAN, JILL MD
<NA>	32.34	0.0	100.0%	ADVANCED HOMECARE
<NA>	300.00	0.0	100.0%	SIGNAL DIAGNOSTICS LLC
<NA>	243.18	0.0	100.0%	ADVANCED HOMECARE
<NA>	300.00	0.0	100.0%	FLOW HEALTH LABORATORIES
<NA>	117.64	0.0	100.0%	ADVANCED HOMECARE
<NA>	75.00	0.0	100.0%	SIGNAL DIAGNOSTICS LLC
<NA>	41.02	0.0	100.0%	ADVANCED HOMECARE
<NA>	75.00	0.0	100.0%	FLOW HEALTH LABORATORIES
<NA>	250.00	0.0	100.0%	GLASER, ELIZABETH B
<NA>	70.38	0.0	100.0%	SIGNAL DIAGNOSTICS LLC
<NA>	250.00	0.0	100.0%	NEUROLOGICAL ASSOCIATES SURGERY CENTER

<NA>	39.75	0.0	100.0%	ADVANCED HOMECARE
<NA>	187.50	0.0	100.0%	NEUROLOGICAL ASSOCIATES SURGERY CENTER
<NA>	117.64	0.0	100.0%	ADVANCED HOMECARE
<NA>	187.50	0.0	100.0%	GLASER, ELIZABETH B
<NA>	243.18	0.0	100.0%	ADVANCED HOMECARE
<NA>	1312.50	0.0	100.0%	NEUROLOGICAL ASSOCIATES SURGERY CENTER

Total alerts: 1851 claims with >50% discount

This report was generated automatically from Amazon Athena healthcare analytics views.

For questions or custom analyses, contact the data engineering team.

 Report saved to s3://hc-pipeline-demo/reports/healthcare_executive_report_20260503_010717.html